Chain of Custody using SHA-256 Hash - Python Demonstration

Python Program:

```
import hashlib
import datetime

def sha256_of(filename):
    with open(filename, "rb") as f:
        return hashlib.sha256(f.read()).hexdigest()

with open("evidence.txt", "w") as f:
    f.write("Suspect chat log: meeting at 10pm, bring the drive.")

seizure_hash = sha256_of("evidence.txt")
custody_log = []
custody_log.append(f"{datetime.datetime.now()} - SEIZED by Officer A -
hash={seizure_hash}")
handoff_hash = sha256_of("evidence.txt")

if handoff_hash == seizure_hash:
    custody_log.append(f"{datetime.datetime.now()} - RECEIVED by Analyst
B - integrity VERIFIED")
else:
    custody_log.append(f"{datetime.datetime.now()} - RECEIVED by Analyst
B - integrity FAILED (tampered!)")

print("=== Chain of Custody Log ===")
for entry in custody_log:
    print(entry)
```

Sample Output:

```
=== Chain of Custody Log ===
2026-07-16 05:27:50.643435 - SEIZED by Officer A -
hash=b7eb05cf41efbcc50adc36b12679ce4f12de58460e59500eb6bb62858810f84c
2026-07-16 05:27:50.644258 - RECEIVED by Analyst B - integrity VERIFIED
```

Explanation:

This program demonstrates the digital forensic chain of custody. An evidence file is created and its SHA-256 hash is calculated when seized by Officer A. During handoff, the file's hash is recalculated. If both hashes match, the evidence integrity is verified, proving that the file has not been altered. If the hashes differ, it indicates possible tampering. SHA-256 hashing helps maintain evidence authenticity throughout an investigation.